pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Classifying 8-K Items with TF-IDF

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: 8-K sections, labelled by their filers

The eight items in the file

1.01	Material definitive agreement
1.03	Bankruptcy or receivership
2.02	Results of operations
2.03	Direct financial obligation
2.04	Triggering events that accelerate
2.06	Material impairments
3.01	Notice of delisting
5.02	Departure or election of officers

Today

1. Codes as text; the file is a sample
2. Split by filing date
3. The heading leak, shown first
4. The classifier on the bodies
5. Rare items, reported on their own
6. Excel: COUNTIFS on text codes
7. AI check: a random split

5,126 sections from 4,965 filings, each labelled by the item its filer chose.

The item the model assigns

An 8-K model's output

Allowed names

the item the model assigns

the model's probability or score in this file

the filer's item

Never called

a default signal

a distress flag or a warning

a credit event or credit view

Every excerpt has its source line. Company names appear only there, as authors. Item 5.02 sections are counted by item; no name from them is printed.

Item codes are text; the file is a sample

```
pd.read_csv(...)["item"]                    # float64 by default: 1.01 as a number
float("2.10")                               # 2.1: a code would lose its last digit
pd.read_csv(..., dtype={"item": str})       # the course reads codes as text
```

580

Item 1.03 sections, bankruptcy

683

Item 2.04 sections, triggering events

363

Item 2.06 sections, the smallest

700

each of the five full items

At most 100 sections per item per year: how rare an item is here is the course's choice.

Each text is the section's body without its heading, cut at 300 words (median 161).

The split by date

Training

3,650 sections

filed 2019 to 2023

fitted on

Validation

741 sections

filed in 2024

scored on; 2.06 has 41

Test

735 sections

filed in 2025

closed until day 10

```
train, valid, test = textkit.date_split(sections, "filed", "2023-12-31", "2024-12-31")
del test                      # closed until day 10: no later cell can score it
```

A company files the same wording quarter after quarter: a random split puts it on both sides.

The heading leak, shown first

```
Item 2.04 Triggering Events That Accelerate or Increase a Direct Financial Obligation ...
```

0.9899

validation macro F1, heading put back

0.9269

validation macro F1 on the stored bodies

The heading is the label, written in the text. The fix: fit on the bodies the tool stored.

Largest coefficients, Item 2.04

`2.04`

`item 2.04`

`accelerate`

`triggering events`

A model that finds the label in its input scores near 1.0 and has learned nothing to keep.

The classifier on the bodies

Item	Precision	Recall	F1	Sections
1.01	0.8585	0.9100	0.8835	100
1.03	0.9778	0.8800	0.9263	100
2.03	0.8660	0.8400	0.8528	100
2.04	0.9020	0.9200	0.9109	100
2.06	0.8810	0.9024	0.8916	41
macro	0.9271	0.9278	0.9269	741

Against the majority item

Every section assigned Item 1.01: macro F1 0.0297, accuracy 0.1350. The model: macro F1 0.9269, accuracy 0.9298.

TF-IDF inside the Pipeline: 40,403 terms, learned from the training rows only.

Macro F1 gives a rare item the weight of a common one. Accuracy appears only beside the baseline.

Where the misses go

Filer's item	1.01	1.03	2.02	2.03	2.04	2.06	3.01	5.02
1.01	91	0	1	5	2	0	0	1
1.03	3	88	0	4	2	2	0	1
2.02	0	0	100	0	0	0	0	0
2.03	8	0	0	84	5	2	0	1
2.04	4	0	0	4	92	0	0	0
2.06	0	2	1	0	0	37	0	1
3.01	0	0	0	0	0	0	100	0
5.02	0	0	1	0	1	1	0	97

The largest miss

8 sections filed as 2.03 are assigned 1.01: a new obligation is often created by an agreement, and the two items share their words.

Columns: the item the model assigns. Codes as text, in one order on both sides.

The rare items, and class weights

0.8800
recall, Item 1.03

0.9200
recall, Item 2.04

0.9024
recall, Item 2.06 (41 sections)

0.9268
Item 2.06, balanced weights

```
textkit.text_classifier(class_weight="balanced")     # each item weighted by 1 / its share
```

The two models assign a different item to 3 of 741 sections. Chosen on validation by macro F1: tfidf_logistic_balanced, 0.9271 against 0.9269. With weights the outputs are scores, not probabilities.

One setting changed: class_weight, None to balanced; chosen on the validation sections.

What the model leans on

2.03 term	Coefficient	2.04 term	Coefficient
item 2.03	3.1172	default	2.9331
2.03	3.0979	2.04	2.4812
item 1.01	2.6280	item 2.04	2.4650
1.01	2.5780	of default	2.1376
note	1.5755	notes	2.1013
item	1.4812	event	1.7156

Bodies naming their own item

Item 2.02: a share of 0.5780. Item 2.03: 0.4420. Cross-references a filer writes, not the heading.

Item numbers removed: macro F1 0.9234 against 0.9269. A coefficient is not a reason.

The model output label, and the log

0.7782

the model's probability for Item 2.04 in this file, one 2024 section

Which of eight headings the text resembles. Never a default signal, a distress flag, or a credit event; nothing about the company.

Model	Macro F1	Accuracy	Recall 1.03	Recall 2.04
majority_item	0.0297	0.1350	0.00	0.00
tfidf_logistic	0.9269	0.9298	0.88	0.92
tfidf_logistic_balanced	0.9271	0.9298	0.88	0.92

results.csv: 4 decimals, LF line endings; the heading model is not a candidate.

Excel side by side

Excel side by side (tested in Excel)

`=COUNTIFS(validation!C2:C742,"2.04",validation!D2:D742,"2.04")`	92
`=COUNTIFS(validation!$C$2:$C$742,$A2,validation!$D$2:$D$742,B$1)`	B2:I9, all equal; 741
`=COUNTIFS(validation!$C$2:$C$742,A2,validation!$D$2:$D$742,A2)/COUNTIF(validation!$C$2:$C$742,A2)`	recall, equal
`=2*J2*K2/(J2+K2)`	F1, within 1e-12
`=AVERAGE(L2:L9)`	0.9271
`=SUMPRODUCT(--ISTEXT(validation!C2:D742))`	1,482 text cells

No tokenizer, TF-IDF, or text classifier in Excel. The results table is all COUNTIFS and AVERAGE.

Codes as text, and what COUNTIF does

Written into	Excel holds	ISTEXT
a General cell: 2.10	the number 2.1	FALSE
a Text cell: 2.10	"2.10", LEN 4	TRUE
pandas to_excel: 2.10	"2.10"	TRUE

Three Text cells

`"2.10", "2.1", "2.10"`

Excel side by side (tested in Excel)

`=COUNTIF(Q1:Q3,"2.10")`	3: "2.1" counted
`=COUNTIF(Q1:Q3,"2.1")`	3
`=SUMPRODUCT(--EXACT(Q1:Q3,"2.10"))`	2, as Python's ==

COUNTIF compares number-like text as numbers. EXACT compares the text.

Save the day: the first results.csv

1. Init

the work folder, settings, the guard

2. Code

mlkit and textkit, one commit

3. Log

results.csv, its own commit

4. History

git log -- results.csv

```
.gitignore      |   5 +
mlkit.py        | 568 ++++++
```

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: a random split: macro F1 0.9339, 40 validation sections share a filing with training; dates 2019-01-03 to 2024-12-30.

Code that runs is not the same as code that is right.

What you can do now

- Read item codes as text, in pandas and Excel
- Split dated filings by date; close the test rows
- Find a heading leak, and fit on the bodies
- Report the rare items' recall on its own
- Build the confusion matrix with COUNTIFS

Week 1, Day 4 Word embeddings